

dash-mno-verify, adversarial code review findings

Date: 2026-06-26 Repository: dash-mno-verify (anonymous zero-knowledge proof of Dash masternode control, used to gate access to private communities)

Reviewers: three independent passes. OpenAI Codex ran a whole-repo audit. Two Claude Code sub-reviews ran in parallel, one on zero-knowledge circuit and nullifier soundness, one on the JavaScript gateway and oracle plus the live uncommitted diff. This review used the new third lens, creative out-of-the-box opportunities kept grounded in the code. Findings reached independently by more than one reviewer are marked (consensus).

Verdict

Consensus BLOCK, and that matches the repo's own "not audited" disclaimer. The cryptographic core is in good shape. The circuits are correctly constrained, hash160 is implemented and continuous- integration-tested against a known vector, the setup is transparent PLONK with no trusted-setup risk, and the verifier hard-fails on an invalid proof. The blockers are in the binding and trust layer around the proof, not in the proof itself. Do not deploy this to gate anything of value yet, and do not commit the current working-tree diff as-is until the season-rollover race is fixed (the new registration store and contract parts of the diff are good).

Lens 1, correctness and security

B1. The proof is bound to a nonce, not to the requesting account, so a valid proof can be relayed to grant a stranger (consensus: all three, BLOCKER)

The challenge stores the `account` server-side, but `signalHash` is a function of the nonce only, the account is never a circuit input and is never re-checked at verify time, and every adapter grants the role to whoever submits a valid `(nonce, proof)` pair (`adapters/discord/bot.js:99,` `adapters/matrix/bot.js:75,` `adapters/telegram/bot.js:73,` `adapters/web/server.js:122`; gateway returns the bound account at `core/gateway.js:176` but no adapter compares it). A proof obtained for account A grants account B if B submits it. The code comment claiming "a proof made for one account cannot grant another" (`core/stores.js:59`) is false as wired. Fix: every adapter must reject unless `out.account` equals the submitter, then grant `out.account`. Stronger fix in Lens 3 (bind the account into the circuit signal).

B2. The two-tier members root is not scoped to context, so one registration grants every community that season (Codex, BLOCKER)

The registration nullifier is context-scoped, but the accepted members root is a single global season tree (`core/gateway.js:71,144,190`, `core/registration_store.js:55`). A user who registers once for any (`platform`, `community`, `role`) in a season can then prove membership against that same root for every other context in that season. This is distinct from B1 and survives a B1 fix. Fix: build and serve members trees per (`season`, `contextHash`), or make the registered leaf `Poseidon(secret, contextHash)` and key roots by context (`forSeason` becomes `forSeasonContext`).

M1. Private-key limbs are not constrained below the secp256k1 group order, so the nullifier is malleable (Sybil: a second access per epoch) (crypto pass, MAJOR)

`ECDSAPrivToPub` range-checks each limb to 64 bits but never constrains the 256-bit scalar `d` to be less than the group order `n`. Since `d` and `d + n` give the same public key and the same `hash160` leaf but a different nullifier (the nullifier is derived from the raw private-key limbs, not the canonical key), one operator can mint a second valid, non-colliding nullifier for the same node in the same epoch. Confidence medium-high; confirm by building two witnesses with scalars `d` and `d + n` and checking both verify with different nullifiers. Fix: add an in-circuit `d < n` check, or derive the nullifier from the in-circuit leaf `hash160(Q)` instead of the raw limbs (see Lens 3).

M2. Season-rollover time-of-check-to-time-of-use race in the register path, introduced by the uncommitted diff (JS pass, MAJOR, borderline blocker)

`verifyRegistration` captures the module-level `membersTree` by reference, then awaits the proof verify and the durable append. During those awaits the 60-second rollover timer or a concurrent challenge can call `rebuildSeason` and reassign `membersTree/membersRoots` (`core/gateway.js:71-86,113,185-196`). On resume the registration appends to the stale tree while the gateway publishes a root from that stale tree into the new season's store, so a published root can diverge from the live tree. The durable record is written under the correct season, so a later rebuild recovers, but within the process the views can disagree for the rest of the root window. Fix: run the whole register critical section inside the season serialization (`seasonOp`), and re-check `currentSeason` immediately before `membersTree.append`, aborting if it moved.

M3. The oracle root is trusted without freshness or source authentication (consensus: Codex + both passes, MAJOR)

The gateway accepts whatever JSON the single configured source returns and records `o.root` with no signature, no schema or format validation, and no recomputation (`core/stores.js:33`, `core/gateway.js:55`; `isRecent` only checks ring-buffer membership

at `core/stores.js:27`). Plain http is allowed, there is no fetch timeout or size cap, and `height` is attacker-settable. A compromised, misconfigured, or man-in-the-middle source can publish a root over an attacker-chosen masternode set and admit a non-masternode. Fix: have the gateway recompute the root from the published leaves (it already ships the tree builder) and reject on mismatch, require https, add a max root age, validate `depth`, `height`, and `leaves.length`, and prefer signed or Platform-published roots. See Lens 3.

M4. Verification-key and public-signal binding is not asserted in continuous integration (crypto pass + Codex, MAJOR)

The committed verification keys are not diffed against keys exported from the deployed circuits in CI, and `core/verifier.js` hard-codes the public-signal index map with a comment saying to confirm it against the compiled circuit's `public.json`, which is never done. A wrong signal-index map paired with a matching key is a real soundness risk, because the gateway would policy-check the wrong public value (for example comparing the proof's epoch slot against the expected `root`). Fix: in CI, export each circuit's vkey and diff it against the committed file, and add one test that runs a known witness and asserts the public-signal ordering matches the verifier's index map.

M5. The challenge and verify endpoints are unauthenticated and unthrottled (JS pass, MAJOR)

`/v1/challenge` mints unlimited live nonces for caller-chosen `platform/community/role/account` with no auth or rate limit and an unbounded in-memory map, and `/v1/verify` runs a full PLONK verify on attacker-supplied input with no rate limit (`core/gateway.js:140-153`). This is both a denial-of-service amplifier and the supply side of the B1 relay (pre-minting nonces for victim accounts). Fix: rate-limit both endpoints, cap the challenge map, and require the adapter to authenticate to the gateway so the `account` is vouched for rather than caller-supplied.

M6. The web adapter session is forgeable and unprotected (JS pass, MAJOR for the web gate)

The session id is an unsigned `randomUUID`, a fresh cookie is emitted on most responses, there is no CSRF protection on the state-changing POSTs, and `/api/submit` marks the current session verified without checking `out.account === sid` (`adapters/web/server.js:94-133`). Fix: check `out.account === sid` before granting, sign the cookie, add CSRF tokens, and only set the cookie on the initial GET.

Minor

- Platform-backed two-tier is documented as available but the gateway aborts for `MNO_MODE=two-tier` plus `MNO_STORE=platform` (`core/gateway.js:91`). The abort

itself is the safe choice (failing loud beats a non-shared store that could double-grant), but the docs overpromise. Fix the docs or implement the backend. (Codex)

- Two-tier commitments are linkable if a user reuses a secret, because the leaf is `Poseidon(secret)` with no domain separation (`circuits/mno_registration.circom:51`). The CLI draws a fresh secret, so this is user-error dependent. Domain-separate with `contextHash`. (Codex)
- No grace window at the epoch boundary, so a challenge minted just before a boundary is rejected (`core/gateway.js:148` vs `:175`). Availability, not security. (JS pass)
- The file-backed registration store serializes appends only within one process; two gateways on the same file can assign duplicate indices. Document single-writer or add a file lock. (JS pass)

Lens 2, architecture and design

- Tree capacity is implicit and unchecked. Both the DML and members trees assume depth 16 (65,536 leaves). Beyond that, the JavaScript builders stop matching the circuit and fail oddly (`oracle/oracle.js:22`, `core/members_tree.js:52`). Reject inserts and snapshots over `2 ** TREE_DEPTH`, and make depth a shared constant. (Codex, MAJOR)
- The registration store's query model bakes in season-only roots, which is the structural source of B2. Make context part of the store's primary read path. (Codex, MINOR)
- Otherwise the architecture is reasonable. Module boundaries (`circuits`, `prover`, `oracle`, `gateway`, `adapters`, `core`) are clean, and the verifier hard-fails with no try/catch swallowing a failure into success and no default-allow.

Verified sound (no action needed)

- `hash160 = RIPEMD160(SHA256(compressed key))` is implemented carefully with correct endianness and is CI-tested end to end against the `secp256k1` generator vector. (crypto pass)
- Merkle inclusion enforces the path-index boolean and root equality, and the Semaphore signal-binding constraint is correct. The circuit-level binding is fine. (crypto pass)
- Transparent PLONK over a shared universal SRS, so no Groth16 toxic-waste exposure. Verification keys are committed and config-pinned, not attacker-supplied at run time. (crypto pass)
- No assign-without-constrain (`<--` only) signals anywhere in the first-party circuits. The classic under-constrained-signal bug is absent. (crypto pass)
- The verifier runs all policy checks and hard-fails on an invalid proof. (JS pass)

The uncommitted diff

- `core/registration_store.js` (new) and the contract change are safe and a genuine improvement. The store makes the registration spend atomic and durable, fixing the prior two-step spend-then-append bug, and it is well tested. It cannot be poisoned by proof content, because the proof is verified before the append.
- `core/gateway.js` season-rollover machinery introduces M2 (the TOCTOU). Fix before committing.
- B1 (account binding) is pre-existing, not introduced by the diff, but the diff edits the very files and docs that claim the binding works, so fix it in the same pass.

Lens 3, creative opportunities (grounded, not fixes)

1. Bind the account into the circuit signal. Set `signalHash = hashToField(nonce + ":" + account)` and have the prover echo it. The proof itself becomes retarget-proof, so a stolen nonce or proof is useless for a different account. This is a one-line change plus passing the account to the prover and is the strongest fix for B1. All three reviewers converged on this. Assumption: the prover can receive the account at proof time (it already receives the challenge).
2. Make the oracle root self-verifying at the gateway. The gateway already ships the Poseidon tree builder and the leaf encoding, so `refreshRoots` can recompute the root from the snapshot's published leaves and reject on mismatch. This turns "anyone can recompute" from a property nobody exercises into one the gateway enforces, neutralizing a malicious oracle without signatures. Effort low to medium. Assumption: snapshots keep publishing the ordered real leaves.
3. Use context-scoped incremental members trees keyed by `(season, contextHash)`. Fixes B2 and avoids rebuilding a full 65,536-leaf tree on every append. Effort medium. Assumption: each gateway can cache a small tree per context.
4. Derive the nullifier from the in-circuit leaf `hash160(Q)` rather than the raw private-key limbs. This kills the group-order malleability of M1 for free, because the leaf is canonical for a given node regardless of which congruent scalar produced it. Effort low (a circuit change plus a re-setup). Assumption: the voting-key anchor is unchanged.
5. Mint a signed, single-use, account-scoped, epoch-scoped grant token on success that the adapter verifies before assigning the role. This separates "a node proved control" (the nullifier) from "this account is the one that proved" (the token), so even a buggy adapter cannot grant to the wrong identity, and it gives the idempotent re-grant the TODO already wants. Assumption: the gateway can hold a signing key. No new circuit work.
6. Add an HTTP-layer integration test harness that boots the real gateway on loopback and asserts the negative paths: wrong account rejected, replayed nonce rejected,

tampered public signals rejected, and a registration straddling a season boundary handled consistently. This is exactly the layer where B1, B2, M2, and the verifier's policy checks live, and it is currently untested.

Coverage and gaps

Covered: all five circuits, both provers, the oracle, the core gateway and verifier and stores and the new registration store, the four adapters, the contract, and the uncommitted diff. The test suite was not executed (the review ran read-only and the tests create temporary files); the unit tests for the stores are good, but there are no tests for the wrong-account rejection, the season rollover, the verifier's HTTP-layer policy checks, or oracle-root trust. Adding those (idea 6) is the highest-value test work.

Recommended next steps (do in this project's session)

- Pre-deploy blockers: B1 account binding and B2 context-scoped roots.
- Before committing the current diff: M2 season-rollover serialization.
- Then M1 nullifier canonicalization, M3 oracle-root self-verification, and M4 through M6 endpoint and web hardening.
- Add the negative-path tests, then run `git review` so a different model re-checks the fixes.

Provenance

This document is the durable, session-independent synthesis. Three reviewers ran on 2026-06-26 (Codex plus two Claude sub-reviews). The Codex run cost 88,689 tokens. The raw reports are transient. There is no FUTURE_DIRECTIONS file in this repo; TODO.md is the natural place to track these items, and it had uncommitted edits at review time, so it was left untouched here.